

从桌面到上线

2026blog 零基础操作手册

给 Gatsby 的第一份网站说明书。你不需要先会编程；只要按顺序完成每一个小动作，就能把自己的主页、简历和笔记空间放到互联网上。

你将学会

打开项目 → 在本机预览 → 上传 GitHub → 部署网站 → 在网页上写笔记

版本 1.0 · 2026-07-17 · 文件夹：桌面\2026blog

先认识这几个东西

先读完这一页再动手。理解它们以后，遇到报错也不会慌。

名称	把它想成什么	你会用它做什么
VS Code	写作桌	打开、修改这个网站里的文件。
终端 Terminal	给电脑下命令的小窗口	运行网站、把改动上传。照着输入即可。
Git	项目的时光机	记录每一次改动，改错了也能找回。
GitHub	网上的项目保险箱	保存代码，供另一台电脑和网站读取。
Cloudflare	网站的公开入口	运行网页写笔记所需的安全服务，并让所有人访问你的网站。

先告诉你结论

你的笔记正文会变成 GitHub 仓库里的一篇 Markdown 文件。GitHub 有新提交后，Cloudflare 会重新构建网站，所以不同电脑都能看到同一份内容。

本手册默认你的 GitHub 用户名是 DSlandhou，项目文件夹是 D:\桌面\2026blog。如果你改了其中一个名字，请把命令中的相应部分一起换掉。

开始前准备：联网；已经登录 GitHub；可以打开 VS Code；不要把任何密码、Client Secret 或 .dev.vars 文件发给别人。

文件夹不是杂物箱， 它是一张项目地图

一个网站不会只靠一份 HTML 文件。现代项目把内容、页面、样式和发布逻辑拆开存放，这叫做关注点分离。它的好处是：改简历时不容易弄坏笔记，改颜色时所有页面会一起变。

关键词	先这样理解
01	src 是源文件：它是你真正编辑的原材料。
02	pages 决定网址：例如 notes 文件夹对应 /notes 页面。
03	content 是内容库：每篇文章是一份独立 Markdown 文件。
04	代码分开不是为了复杂，而是为了让未来的你敢于修改。

把它记成一句话
代码分开不是为了复杂，而是为了让未来的你敢于修改。

从桌面打开项目

先让你看见已经生成的网站文件。

01 回到桌面

按键盘 Windows 键，或点击屏幕底部中间的 Windows 图标。点击“桌面”，找到名为 2026blog 的文件夹。

02 进入文件夹

双击 2026blog。你会看到 src、public、package.json 等文件。它们一起组成网站；现在不需要理解每一个。

03 用 VS Code 打开

在文件夹空白处点右键。若菜单有“通过 Code 打开”，点击它。没有时：先打开 VS Code → 左上角 File（文件）→ Open Folder（打开文件夹）→ Desktop（桌面）→ 2026blog → Select Folder（选择文件夹）。

04 相信这个项目

如果 VS Code 顶部弹出“Do you trust the authors...”提示，点击“是，我信任作者”。这是你自己电脑上的文件夹。

05 认准左侧文件树

VS Code 左侧第一个图标像两张纸，叫 Explorer。展开 src，可以看到 pages（网页）、content（笔记内容）、data（简历资料）和 styles（外观）。

不要害怕

真正需要你长期手动编辑的只有两类文件：src/data/profile.ts（简历资料）与 src/content/notes（笔记）。其他文件现在先不要改。

第一次在本机预览

这一步不会上传任何东西，只让你在自己的电脑上看网站。

01 打开终端

在 VS Code 顶部菜单点击 Terminal（终端）→ New Terminal（新建终端）。也可以按 Ctrl + `（反引号，通常在键盘左上角 Esc 下方）。屏幕下方会出现一个可输入文字的区域。

02 确认位置

终端提示文字的最后应该能看到 2026blog。若没有，请停止，重新用“打开文件夹”方式打开 D:\桌面\2026blog。

03 启动网站

在终端复制或手动输入下面这一行，按 Enter。第一次可能需要等半分钟。

```
npm run dev
```

04 打开浏览器

终端出现类似 `http://localhost:4321/` 的蓝色地址时，按住 Ctrl 点击它；或复制地址到 Chrome 地址栏打开。看到 DS、简历、笔记、Now，就成功了。

05 结束预览

看完后回到 VS Code 下方终端，按 Ctrl + C 一次。如果询问是否终止，输入 Y 再按 Enter。下次预览仍然只需 `npm run dev`。

如果网页打不开

先确认终端没有红色报错；然后刷新浏览器。仍不行时，关闭终端重新打开，再执行 `npm run dev`。不要双击 `index.astro` 文件来打开，它不是普通 HTML 网页。

npm run dev 到底做了什么？

浏览器不能直接理解 .astro、TypeScript 等开发文件。Node.js 是电脑上帮助你处理这些文件的运行环境；npm 是它的包裹管理员；dev 命令则临时启动一台只属于你电脑的小型网站服务器。

关键词	先这样理解
01	Node.js 让 JavaScript 能在你的电脑上运行，而不只在浏览器里运行。
02	npm install 把项目依赖下载安装到 node_modules；不要手动修改那个文件夹。
03	localhost 的意思是本机，别人无法通过这个地址看到你的网页。
04	开发服务器会观察文件变化，所以你保存后浏览器能快速刷新。

把它记成一句话
开发服务器会观察文件变化，所以你保存后浏览器能快速刷新。

先把它改成你自己的主页

每次修改后保存，然后刷新浏览器就能看到变化。

修改简历：在左侧打开 `src → data → profile.ts`。把 `name`、`email`、`headline`、`introduction`、`currentFocus` 里的文字改成你的真实内容。每一行文字两侧的引号必须保留；修改后按 `Ctrl + S` 保存。

修改首页视觉：统一外观在 `src → styles →`

`global.css`。现在先不用动它；以后想改颜色、字号、间距时，在这个文件里改，所有页面会一起更新。

手动新增一篇笔记：在 `src → content → notes` 文件夹上点右键 → `New File`（新建文件），文件名写成类似 `2026-07-18-my-first-note.md`。然后复制下面模板：

```
---
title: "■■■■■■■■"
description: "■■■■■■■■■■■■■■■■■■■■"
publishedAt: 2026-07-18
tags: ["■■■", "■■■"]
draft: false
---

■■■■■■■■■■■■■■■■■■■■
```

保存后刷新浏览器的“笔记”页面，新文章就会出现。文件名最好用英文、小写和连字符，标题和正文可以放心写中文。

一个重要习惯

每次只做一个小改动，然后 `Ctrl + S`

保存、刷新浏览器确认。这样出问题时，你永远知道是刚刚哪一步造成的。

Markdown 是什么？为什么笔记不用 Word？

Markdown 是一种轻量的纯文本格式：你看到的是文字，电脑也能根据少量符号把它排成网页。它比 Word 简单、可搜索、能长期保存，也不会被某一个软件锁住。

关键词	先这样理解
01	--- 包起来的部分叫 frontmatter，存标题、日期、标签等资料。
02	# 是一级标题，## 是二级标题，- 是列表，**文字** 是加粗。
03	正文和格式说明都在同一个 .md 文件，因此容易备份和迁移。
04	你今天写的笔记，十年后仍可用任何文本编辑器打开。

把它记成一句话
你今天写的笔记，十年后仍可用任何文本编辑器打开。

第一次上传到 GitHub

这一步把本机文件放进你的 GitHub 账号。只需要做一次。

01 在 GitHub 创建空仓库

浏览器打开 github.com 并登录 DSlandhou。右上角 + → New repository。Repository name 输入 2026blog，选择 Public。不要勾选 Add a README、.gitignore 或 license，然后点击 Create repository。保持这个页面打开。

02 回到 VS Code 终端

确认终端位置是 D:\桌面\2026blog。依次复制下面每一行，粘贴后按 Enter；一行执行完再执行下一行。第一、二行的名字和邮箱请换成你的。

```
git config --global user.name "Gatsby"
git config --global user.email "■■■■■■■■@example.com"
git init
git add .
git commit -m "feat: first version of my blog"
git branch -M main
git remote add origin https://github.com/DSlandhou/2026blog.git
git push -u origin main
```

03 允许 Git 登录

最后一行可能打开浏览器让你登录 GitHub 或授权 Git Credential Manager。确认账号是 DSlandhou 后点允许。回到 VS Code，看到 main → main 或类似成功文字即可。

04 检查云端

刷新刚才 GitHub 的仓库页面。应该能看到 src、package.json、README.md 等文件。以后任何电脑都能从这里取回项目。

如果提示 remote origin already exists

说明这个文件夹曾经连接过 GitHub。不要反复执行 remote add。先在终端执行 git remote -v，把结果复制给我。

Git 不是上传工具， 而是可回退的历史

Git 把项目的变化记录成一系列快照。每一个 commit 都像给当前项目拍一张有说明的照片；GitHub 只是保存这些照片的远程位置。

关键词	先这样理解
01	git add 是挑选哪些改动要放进下一张照片。
02	git commit 是按下快门，并写明这次我改了什么。
03	git push 是把本机照片传到 GitHub；git pull 是把云端新照片取回电脑。
04	小而清晰的提交，会让你未来修复问题和回顾成长都更容易。

把它记成一句话
小而清晰的提交，会让你未来修复问题和回顾成长都更容易。

把网站公开到互联网上

使用 Cloudflare 的免费服务，因为“网页里写笔记并安全提交 GitHub”需要服务器。

01 注册或登录 Cloudflare

浏览器打开 dash.cloudflare.com，使用 GitHub 或邮箱注册并登录。新用户按照屏幕提示验证邮箱即可。

02 在终端登录 Cloudflare

回到 VS Code 下方终端，输入下面命令并按 Enter。浏览器会打开授权页面；确认是你的 Cloudflare 账号后点击 Allow（允许）。

```
npx wrangler login
```

03 第一次部署

浏览器提示成功后回到终端，执行下面命令。首次会花一两分钟。完成后终端会显示一个 <https://...workers.dev> 地址，复制并在浏览器打开。

```
npm run deploy
```

04 保存网址

把这个 [workers.dev](https://...workers.dev) 地址记在安全的地方；下一章需要它。也可以在 Cloudflare 控制台的 Workers & Pages 中找到名为 `gatsby-notes` 的项目。

为什么不是 GitHub Pages？

GitHub Pages 适合纯静态网页，但无法安全保存 GitHub 登录密钥。这里的 Cloudflare Worker 只负责登录和提交，博客内容本身仍保存在你的 GitHub 仓库。

部署不是上传文件， 而是把源码变成网站

当你执行 `npm run deploy`，电脑先 build：把 Astro 页面、样式和文章整理成浏览器能访问的产物；然后 Cloudflare 把它放到全球可访问的服务器。访问者看到的是产物，不是你的 VS Code。

关键词	先这样理解
01	构建（build）把开发用的文件转成适合线上运行的文件。
02	域名或 <code>workers.dev</code> 地址是读者找到服务器的门牌号。
03	这次项目使用服务器渲染：页面之外，还有小型 API 负责安全发布笔记。
04	源码仍在 GitHub，线上部署随时可以重新生成，不怕服务器丢文件。

把它记成一句话
源码仍在 GitHub，线上部署随时可以重新生成，不怕服务器丢文件。

让“写一条”按钮真正能发布

这一步只需配置一次。它让网站确认“正在发布的人就是你”。

01 创建 GitHub OAuth App

GitHub 右上角头像 → Settings → 最底部 Developer settings → OAuth Apps → New OAuth App。Application name 写 Gatsby Notes。Homepage URL 填上一章保存的 workers.dev 网站地址。Authorization callback URL 填：你的 workers.dev 地址/api/auth/callback。点击 Register application。

02 得到两把钥匙

注册完成后复制 Client ID。点击 Generate a new client secret，按提示确认后立刻复制 Client Secret。Client Secret 只显示一次，不要放进记事本、截图或 GitHub。

03 把钥匙安全放进 Cloudflare

回到 VS Code 终端，下面的每个命令都要单独执行。终端会要求你粘贴值；粘贴后按 Enter。输入密码时屏幕不显示字符是正常的。

```
npx wrangler secret put GITHUB_CLIENT_ID
npx wrangler secret put GITHUB_CLIENT_SECRET
npx wrangler secret put GITHUB_REPOSITORY
npx wrangler secret put GITHUB_BRANCH
npx wrangler secret put ALLOWED_GITHUB_LOGIN
npx wrangler secret put BLOG_SESSION_SECRET
```

依次填入：GITHUB_CLIENT_ID = Client ID；GITHUB_CLIENT_SECRET = Client Secret；GITHUB_REPOSITORY = DSlandhou/2026blog；GITHUB_BRANCH = main；ALLOWED_GITHUB_LOGIN = DSlandhou。最后一个 BLOG_SESSION_SECRET 需要随机字符串，下一页告诉你怎么生成。

OAuth：为什么不用把密码直接交给自己的网站？

OAuth 是请 GitHub 临时替你确认身份的授权方式。你在 GitHub 的官方页面登录和点击同意；你的网站只得到一张有限权限的通行证，而看不到你的 GitHub 密码。

关键词	先这样理解
01	Client ID 像应用的公开名字，可以出现在配置中。
02	Client Secret 像应用的私密印章，只能放 Cloudflare Secrets，不能进 GitHub。
03	OAuth token 是短暂通行证，网站用它在你的授权范围内创建笔记文件。
04	安全的原则是最小权限：本项目只请求公开仓库写入，并限制为你的账号。

把它记成一句话
安全的原则是最小权限：本项目只请求公开仓库写入，并限制为你的账号。

生成安全密钥并测试发布

不要跳过随机密钥这一步，它保护你的登录会话。

01 生成随机密钥

在 PowerShell 或 VS Code 终端输入下面三行。它会输出一串很长的随机文字；复制这一整串。

```
$bytes = New-Object byte[] 48  
[System.Security.Cryptography.RandomNumberGenerator]::Fill($bytes)  
[Convert]::ToBase64String($bytes)
```

02 保存随机密钥

执行 `npx wrangler secret put BLOG_SESSION_SECRET`，把刚才复制的随机文字粘贴进去并按 Enter。

03 再部署一次

配置密钥后执行下面命令，让线上的网站读取新配置。

```
npm run deploy
```

04 测试“写一条”

打开你的 `workers.dev` 网站，点击右上角“写一条”→连接 GitHub。确认授权范围是 `public_repo` 后点 Authorize。回到写作页，写一条测试笔记，点击“发布到 GitHub”。

05 确认结果

页面显示“已提交到

GitHub”后，等待约一分钟，刷新网站“笔记”页。文章出现即代表完整流程成功。GitHub 仓库的 `src/content/notes` 里也会多出一个 `.md` 文件。

关于隐私

发布到这个 Public GitHub 仓库的笔记，任何人都可以读到。页面中的草稿只存在当前浏览器；换电脑后不会自动带过去，只有点击发布后的文章会同步。

点击发布后，一篇笔记走了哪条路？

你在写作页输入文字后，浏览器会把它发送给同域名的安全接口。接口检查登录身份、生成 Markdown、调用 GitHub API 创建文件。随后 Cloudflare 根据新提交重新部署，文章才出现在 /notes。

关键词	先这样理解
01	浏览器只负责输入和展示；它永远不保存 Client Secret。
02	服务器负责验证身份和调用 GitHub；它是保护密钥的边界。
03	GitHub 是内容的唯一真实来源，所以两台电脑看到的文章会一致。
04	草稿存在浏览器是为了随手记录；发布后才成为可同步、可访问的正式文章。

把它记成一句话
草稿存在浏览器是为了随手记录；发布后才成为可同步、可访问的正式文章。

以后的日常操作

你不需要每次都碰代码。大多数时候只需打开网站写笔记。

你想做的事	最简单的操作
随手写一条公开笔记	打开网站 → 写一条 → 如有需要先连接 GitHub → 写标题、标签、正文 → 发布到 GitHub → 等待约一分钟。
在本机改简历	用 VS Code 打开 2026blog → 修改 src/data/profile.ts → Ctrl+S → npm run dev 看效果 → 按下面 Git 三步上传。
新增一篇复杂笔记	用“写一条”最快；需要特殊排版时，新建 src/content/notes 里的 .md 文件并写 Markdown。
换另一台电脑	安装 Git、Node.js 和 VS Code → git clone 仓库地址 → npm install → npm run dev。不要复制 node_modules 文件夹。

本机改完代码后的 Git 三步：先确保 npm run dev 已经看过效果，再在终端逐行执行：

```
git add .
git commit -m "docs: update my profile"
git push
```

引号中的英文是这次改动的说明，可按实际内容改。若改的是笔记，可以写 notes: add xxx。

每次开始编辑前
如果你在另一台电脑也改过文件，先在终端运行 git pull，再开始修改。这样能降低“两个版本打架”的机会。

常见问题与求助方式

报错并不代表你做错了，通常只是电脑需要一条更准确的指令。

现象	先做什么
npm run dev 报错	确认终端路径末尾是 2026blog；关闭终端重开，再运行 npm install，最后重试 npm run dev。
Git push 被拒绝	先运行 git pull，再运行 git push。若仍有红色文字，不要乱输命令，把完整红字复制给我。
连接 GitHub 显示配置错误	回到第 6 - 7 章，确认六个 Cloudflare secret 都已设置，尤其仓库名、用户名和 Client Secret。设置后再次 npm run deploy。
发布后文章没出现	先点击页面给出的 GitHub 提交链接确认文件已创建，再等 1 - 2 分钟刷新网站。若 GitHub 没有新文件，检查登录账号。

向我求助时，请一次发三样东西：1) 你刚刚输入的命令；2) 终端里完整的红色报错（文字优先，不要只截一半图）；3) 你期待发生什么。不要发 Client Secret、随机密钥、.dev.vars 或任何 Token。

你不需要一夜之间变成厉害的程序员。

现在只要做第一件事：从桌面打开 2026blog，然后运行 npm run dev。以后每一条写下来的笔记，都会是你重新掌控生活的一个可见证据。